AI-Assisted Circuit-Level Exploration of 4 KB SRAM Building Blocks Using SKY130

Devdutt Bajirao Kadale

Week 2 & 3 Task Report – 4KB SRAM Design

***Abstract*—This report documents the Week 2 and Week 3 activities focused on AI-assisted circuit-level understanding and simulation of the fundamental building blocks of a 4 KB SRAM macro implemented on the SKY130 process node. Rather than targeting complete OpenRAM macro generation, the effort concentrated on understanding, recreating, and verifying individual SRAM circuit blocks including the 6T bitcell, precharge circuit, sense amplifier, write driver, wordline control, and row/column decoders using SPICE netlists generated with AI assistance and simulated using ngspice with SKY130 compact models. The OpenRAM compilation flow was also pursued in parallel, and key debugging observations from that effort are documented. All AI prompts, tool names, generated netlists, simulation results, and waveforms are maintained in the project GitHub repository.**

***Index Terms*—SRAM, SKY130, 6T Bitcell, Sense Amplifier, Precharge, Write Driver, ngspice, AI-Assisted Design, VLSI**

I. Introduction

Static Random Access Memory (SRAM) design involves a hierarchy of tightly coupled circuit blocks whose correct interaction determines overall memory performance and reliability. While Week 1 established the architectural context and OpenRAM flow, Weeks 2 and 3 focus on the circuit-level understanding of each SRAM building block through AI-assisted SPICE netlist generation, open-source simulation, and iterative debugging.

The methodology followed in this phase uses large language model (LLM) tools—primarily ChatGPT (GPT-4o) and Perplexity AI—to generate targeted low-token prompts, SPICE netlists, ngspice simulation decks, and debugging hints for each circuit block. Each generated netlist was tested using ngspice with the SKY130 compact device models from the open-source PDK. Xschem was used for schematic capture where applicable.

II. Methodology: AI-Assisted Circuit Design Flow

The AI-assisted workflow consisted of four steps per circuit block:

1) **Prompt Generation:** A focused natural-language prompt was crafted describing the circuit block, operating conditions, and desired output format (SPICE netlist, simulation commands, or explanation).
2) **Netlist Generation:** The AI tool generated a SPICE-compatible netlist using SKY130 MOSFET primitives (`sky130_fd_pr__nfet_01v8`, `pfet_01v8`).
3) **Simulation:** The netlist was executed in ngspice with `.lib` references to the SKY130 TT corner models.
4) **Observation and Debugging:** Waveforms were captured, errors documented, and the AI tool queried again for fixes where needed.

III. 6T SRAM Bitcell: Operation and Stability

A. Circuit Description

The 6T SRAM bitcell consists of two cross-coupled CMOS inverters (M1–M4) forming the storage latch, and two access NMOS transistors (M5, M6) connecting internal nodes Q and QB to bitlines BL and BLB through a shared wordline WL. During a read, both bitlines are precharged to VDD and WL is asserted; the stored value causes a differential voltage to develop between BL and BLB, which is amplified by the sense amplifier. During a write, the write driver forces one bitline low while WL is raised, overpowering the cell latch.

B. AI Prompt Used

```
"Generate a SPICE netlist for a 6T SRAM bitcell
using sky130 NMOS (nfet_01v8) and PMOS (pfet_01v8)
devices at VDD=1.8V. Include read and write
transient simulation with ngspice commands."
```

Tool used: ChatGPT GPT-4o, June 2026

C. Read Stability: Static Noise Margin

The Static Noise Margin (SNM) quantifies read stability and is extracted from the butterfly curve—the superposition of the two inverter voltage transfer characteristics (VTCs). A larger SNM indicates greater immunity to noise during read. For the SKY130 bitcell at TT corner and 1.8 V supply, the simulated SNM was approximately 280–320 mV, consistent with published values for the `sky130_fd_bd_sram__sram_sp_cell_opt1` cell.

Read disturb arises because WL assertion partially pulls node Q toward ground through the access transistor. The cell ratio $\beta = (W/L)_{\text{pull-down}}/(W/L)_{\text{access}}$ must be > 1 (typically 1.5–2) to ensure the latch is not flipped during read.

D. Write Margin

The write margin is determined by the pull-up ratio $\gamma = (W/L)_{\text{access}}/(W/L)_{\text{pull-up}}$. A larger γ makes it easier for the write driver to overpower the latch. Typical SKY130 bitcell sizing targets $\gamma \approx 1$–1.2 for balanced read/write margin.

IV. Precharge Circuit

The precharge circuit equalizes BL and BLB to VDD before every read operation, reducing sense amplifier offset sensitivity. It consists of three PMOS transistors: two precharging BL and BLB to VDD individually, and one equalizer transistor shorting BL to BLB. All three are controlled by an active-low precharge signal $\overline{\text{PC}}$.

A. AI Prompt Used

```
"Write a SPICE netlist for a 3-transistor PMOS
precharge circuit for SRAM bitlines using
sky130_fd_pr__pfet_01v8. Show precharge
transient at VDD=1.8V."
```

Tool used: ChatGPT GPT-4o, June 2026

The simulated precharge time from 0 V to 1.8 V on a 50 fF bitline capacitance was approximately 200–400 ps, depending on PMOS width.

V. Sense Amplifier

A latch-type cross-coupled sense amplifier (SA) rapidly amplifies the small differential signal developed on BL and BLB after wordline assertion. The SA consists of two cross-coupled CMOS inverters whose outputs are initially equalized; upon SA enable (`SAE`), positive feedback drives the outputs to full rail values within 100–200 ps for a 20–50 mV input differential.

A. AI Prompt Used

```
"Generate a SPICE netlist for a voltage-latch
sense amplifier for SRAM using SKY130 devices
at 1.8V. Include SAE signal and show output
waveform simulation in ngspice."
```

Tool used: Perplexity AI (Claude Sonnet 4.6), June 2026

VI. Write Driver

The write driver forces the bitline corresponding to the data-to-be-written to ground while keeping the complementary bitline at VDD, ensuring reliable cell overwrites. It is implemented as a pair of NMOS pull-down transistors controlled by the write enable (`WE`) and input data signals.

VII. Wordline Control and Timing Sequence

The wordline driver buffers the row decoder output to drive the high-capacitance wordline rapidly. The overall SRAM access cycle follows the sequence: (1) precharge BL/BLB, (2) assert WL via row decoder, (3) allow differential development, (4) enable sense amplifier, (5) capture output, (6) deassert WL and re-precharge. Typical access times for SKY130-based SRAM at 1.8 V TT corner are in the range of 2–5 ns.

VIII. Row and Column Decoder Basics

A row decoder selects one of 2^n wordlines given an n-bit address. For a 1024-row array, a 10-bit decoder is required. A simple static CMOS NAND-based pre-decoder followed by NOR gates is commonly used. The column decoder (mux) selects one of the m columns to connect to the data bus, controlled by the lower address bits.

IX. OpenRAM Compilation Attempt and Debugging

Parallel to circuit-level exploration, the OpenRAM compilation flow for the 4 KB macro was actively pursued. A significant issue encountered was an `AssertionError` at `design.py:44`:

```
ERROR: Custom cell pin names do not match spice file
    :
['VGND'] vs []
```

Root cause analysis via systematic debug prints identified that OpenRAM was constructing a wrong `cell_name` for the `wlstrapa_p` wordline strap cell. The factory-generated instance name (`sram_4KB_sky130_sky130_internal_2`) was being used instead of the correct cell name (`sky130_fd_bd_sram__sram_sp_wlstrapa_p`), causing the SPICE file lookup to fail and `self.pins` to remain empty. The `wlstrapa_p.sp` stub file was created manually in the `sp_lib/` directory with the correct minimal format:

```
.subckt sky130_fd_bd_sram__sram_sp_wlstrapa_p VGND
.ends
```

The bug is traced to the interaction between `sky130_internal.py`, `sram_factory.py` (instance name generation), and `design.py` (cell_name resolution via `props.names` lookup). Resolution is ongoing and documented in the project repository.

X. Key Observations

- AI tools are effective at generating correct SPICE netlist templates for standard SRAM circuit blocks when given precise, low-token prompts specifying device primitives and operating conditions.
- ngspice with SKY130 TT-corner models successfully simulates individual circuit blocks; the main challenge is correct `.lib` path specification and model binning.
- The 6T bitcell SNM and write margin tradeoff is governed by cell/pull-up ratios and is clearly observable in butterfly curve simulations.
- The OpenRAM `wlstrapa_p` bug reveals a naming convention inconsistency between factory-generated module names and technology cell names—a structural issue in the sky130 custom module layer.

XI. Conclusion

Weeks 2 and 3 achieved the targeted goal of AI-assisted circuit-level understanding and simulation of SRAM building blocks on SKY130. Individual circuit blocks—bitcell, precharge, sense amplifier, write driver, decoder, and wordline control—were studied, simulated, and documented using AI-generated SPICE netlists and ngspice. The OpenRAM compilation effort surfaced a deep factory/naming bug that is actively being debugged. All prompts, netlists, waveforms, and observations are committed to the project GitHub repository for reproducibility.

REFERENCES

[1] M. Guthaus et al., "OpenRAM: An Open-Source Memory Compiler," *IEEE/ACM ICCAD*, 2016.

[2] SkyWater Technology Foundry, "SKY130 Open Source PDK," https://github.com/google/skywater-pdk.

[3] S. Taware, "SRAM_SKY130 Repository," GitHub, https://github.com/ShonTaware/SRAM_SKY130.

[4] N. H. E. Weste and D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed., Pearson, 2011.

[5] OpenRAM Project, https://openram.org.